



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
"МИРЭА — Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
по дисциплине
«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Боргачев Т. М.

Принял доцент кафедры ЦТ

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №5. ОБЪЕКТЫ БАЗЫ ДАННЫХ: ПРЕДСТАВЛЕНИЯ И ХРАНИМЫЕ ПРОЦЕДУРЫ

Цель: формирование у студентов углубленных навыков работы с объектами баз данных в СУБД PostgreSQL, смещая акцент от прямого манипулирования данными к созданию переиспользуемых логических конструкций.

Постановка задачи:

Задание №1: создание модифицируемого представления.

Для вашей базы данных создать простое модифицируемое представление, которое отбирает строки из одной таблицы по определенному критерию. Например, для БД «Аптека» можно создать представление, отображающее лекарства только от одного производителя «Bayer AG»).

Задание №2: модификация данных через представление.

Продемонстрировать возможность изменения данных в базовой таблице через представление, созданное в Задании №1. Для этого необходимо выполнить два запроса:

1. Добавить новую запись с помощью оператора INSERT.
2. Удалить существующую запись с помощью оператора DELETE.

Задание №3: создание немодифицируемого аналитического представления.

Для вашей базы данных создать единое немодифицируемое представление для аналитических целей. Представление должно объединять данные как минимум из двух таблиц и содержать агрегирующие функции (COUNT, SUM, AVG и т.д.) и группировку (GROUP BY). Например, для БД «Аптека» такое представление могло бы для каждого производителя выводить общее количество наименований лекарств и их среднюю цену.

Задание №4: использование аналитического представления в запросах.

Написать SELECT-запрос, который использует созданное в Задании №3 аналитическое представление в качестве источника данных для дальнейшей фильтрации или анализа. Например, можно отобрать производителей, у которых средняя цена на продукцию превышает определенное значение.

Задание №5: Создание и обновление материализованного представления.

1. Создать материализованное представление для ускорения выполнения ресурсоемкого аналитического запроса. Например, для БД «Аптека» можно создать представление, которое заранее рассчитывает общую сумму продаж для каждого покупателя.
2. Продемонстрировать процесс обновления данных в представлении с помощью команды REFRESH MATERIALIZED VIEW viewName;

Задание №6: разработка пользовательской функции для аналитических

вычислений.

1. Разработать пользовательскую функцию, которая инкапсулирует комплексный аналитический расчет. Функция должна принимать на вход идентификатор (например, `manufacturer_id`) и возвращать одно скалярное значение (например, общую сумму продаж продукции данного производителя), вычисленное на основе соединения нескольких таблиц и применения агрегатных функций.
2. Продемонстрировать вызов функции в составе SELECT-запроса.

Задание №7: разработка хранимой процедуры для выполнения сложной операции.

Разработайте хранимую процедуру, которая выполняет безопасную операцию по изменению данных. Процедура должна принимать на вход ID какой-либо записи и числовое значение (например, количество). Внутри процедуры необходимо проверить, достаточно ли текущего значения в числовом поле одной таблицы для выполнения операции.

- Если да – уменьшите это значение и добавьте новую запись в другую, связанную таблицу.
- Если нет – операция должна полностью прерваться, не внося никаких изменений в данные.

Для сообщения о результате используйте выходной параметр, который вернёт статус успеха или неудачи.

Задание №8: демонстрация вызова хранимой процедуры.

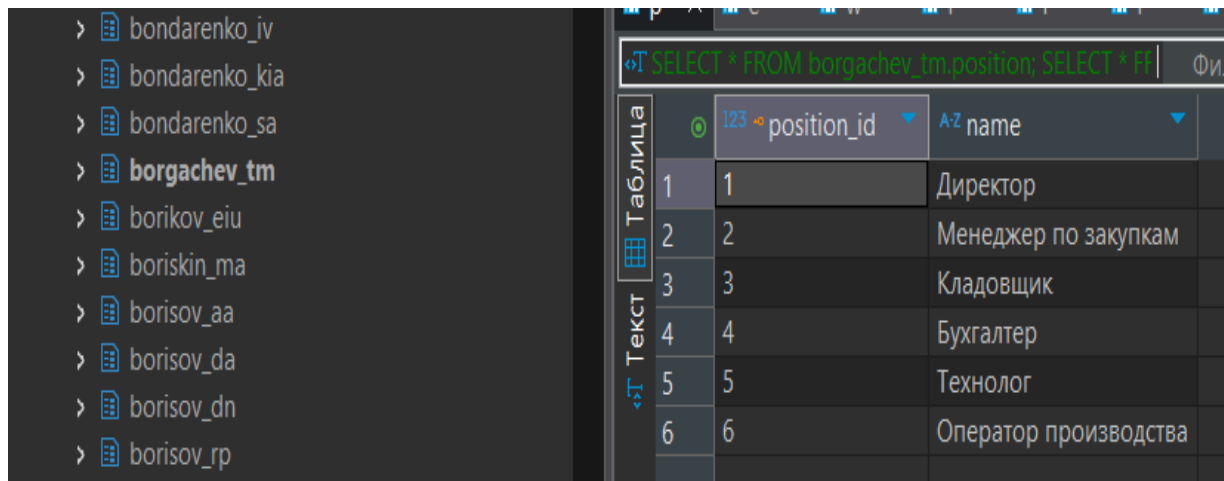
Привести два примера вызова процедуры, созданной в Задании №7:

- Успешный вызов, который добавляет в вашу базу данных уникальную запись.
- Неудачный вызов, который демонстрирует срабатывание реализованной проверки целостности и возврат пользовательской ошибки. Каждый SQL-запрос сопровождать комментарием, объясняющим его назначение и логику работы с учетом специфики вашей базы данных.

ХОД РАБОТЫ

Задание №1: создание модифицируемого представления

Таблица position представлена на Рисунке 1.



position_id	name
1	Директор
2	Менеджер по закупкам
3	Кладовщик
4	Бухгалтер
5	Технолог
6	Оператор производства

Рисунок 1 —Таблица position

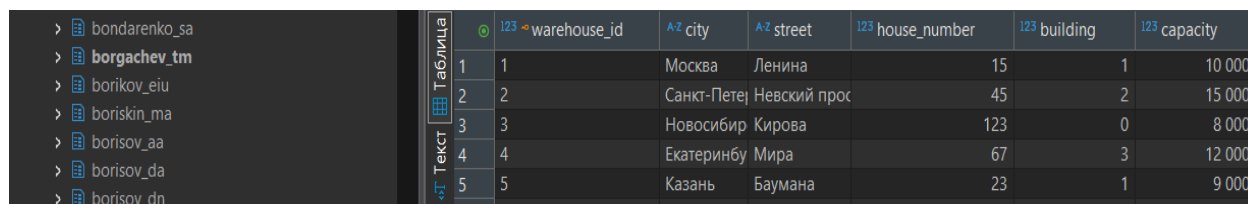
Таблица employee представлена на Рисунке 2.



employee_id	position_id	name	last_name	patronymic	work_years	manager_id
1	1	Иван	Петров	Сергеевич	10	[NULL]
2	8	Виктор	Смирнов	Петрович	2	[NULL]
3	3	Алексей	Сидоров	Викторович	3	2
4	4	Елена	Козлова	Дмитриевна	1	2
5	6	Сергей	Волков	Петрович	4	2
6	2	Мария	Иванова	Александровна	5	1
7	5	Дмитрий	Морозов	Андреевич	6	1
8	7	Ольга	Новикова	Владимировна	2	1

Рисунок 2 —Таблица employee

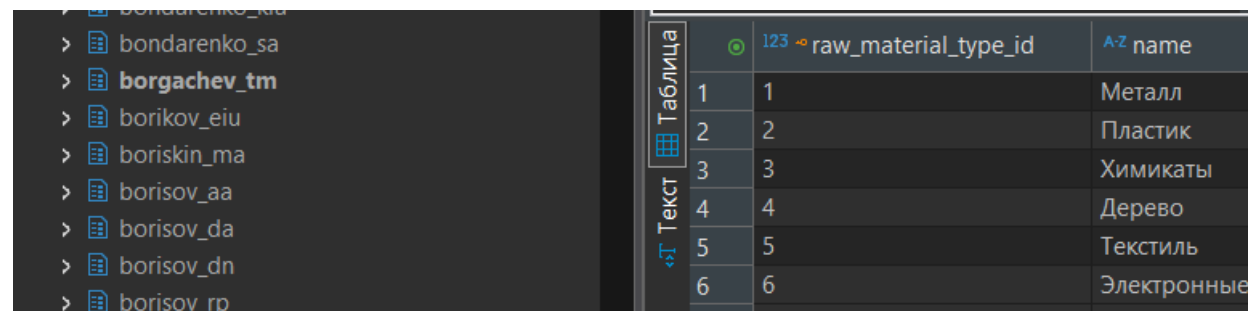
Таблица warehouse представлена на Рисунке 3.



warehouse_id	city	street	house_number	building	capacity
1	Москва	Ленина	15	1	10 000
2	Санкт-Петербург	Невский прот	45	2	15 000
3	Новосибирск	Кирова	123	0	8 000
4	Екатеринбург	Мира	67	3	12 000
5	Казань	Баумана	23	1	9 000

Рисунок 3 —Таблица warehouse

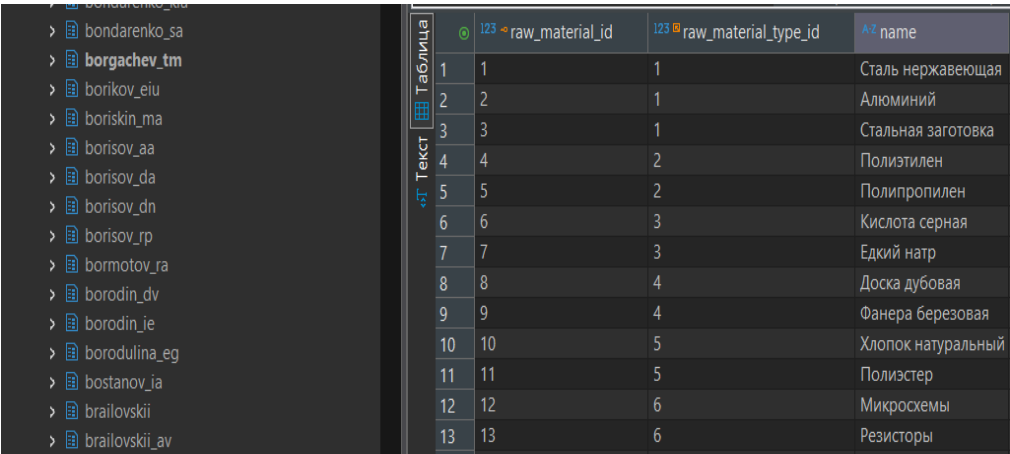
Таблица raw_material_type представлена на Рисунке 4.



raw_material_type_id	name
1	Металл
2	Пластик
3	Химикаты
4	Дерево
5	Текстиль
6	Электронные

Рисунок 4 —Таблица raw_material_type

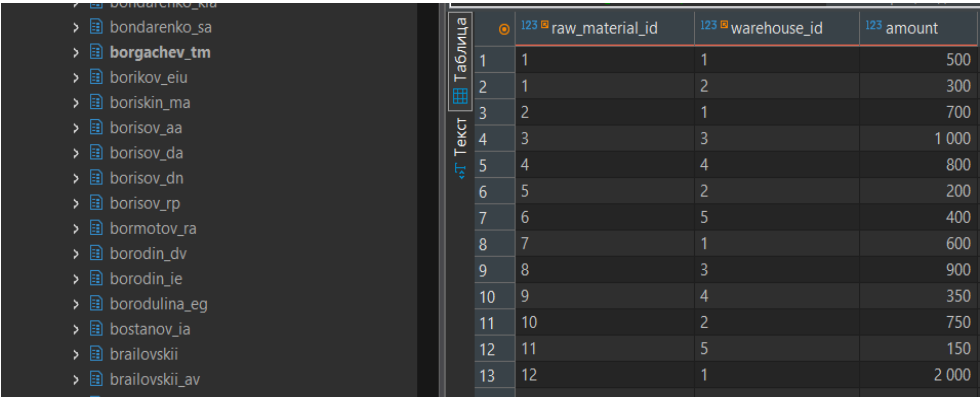
Таблица raw_material представлена на Рисунке 5.



	123 raw_material_id	123 raw_material_type_id	A-Z name
1	1	1	Сталь нержавеющая
2	2	1	Алюминий
3	3	1	Стальная заготовка
4	4	2	Полиэтилен
5	5	2	Полипропилен
6	6	3	Кислота серная
7	7	3	Едкий натр
8	8	4	Доска дубовая
9	9	4	Фанера березовая
10	10	5	Хлопок натуральный
11	11	5	Полиэстер
12	12	6	Микросхемы
13	13	6	Резисторы

Рисунок 5 —Таблица raw_material

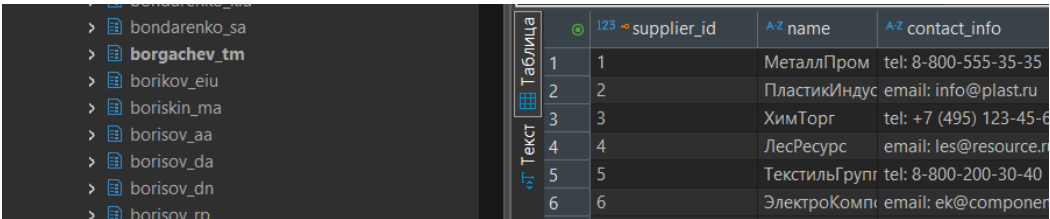
Таблица inventory представлена на Рисунке 6.



	123 raw_material_id	123 warehouse_id	123 amount
1	1	1	500
2	1	2	300
3	2	1	700
4	3	3	1 000
5	4	4	800
6	5	2	200
7	6	5	400
8	7	1	600
9	8	3	900
10	9	4	350
11	10	2	750
12	11	5	150
13	12	1	2 000

Рисунок 6 —Таблица inventory

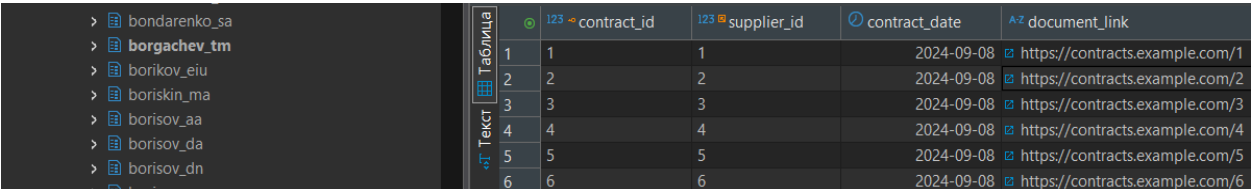
Таблица supplier представлена на Рисунке 7.



	123 supplier_id	A-Z name	A-Z contact_info
1	1	МеталлПром	tel: 8-800-555-35-35
2	2	ПластикИндус	email: info@plast.ru
3	3	ХимТорг	tel: +7 (495) 123-45-6
4	4	ЛесРесурс	email: les@resource.ru
5	5	ТекстильГрупп	tel: 8-800-200-30-40
6	6	ЭлектроКомп	email: ek@componen

Рисунок 7 —Таблица supplier

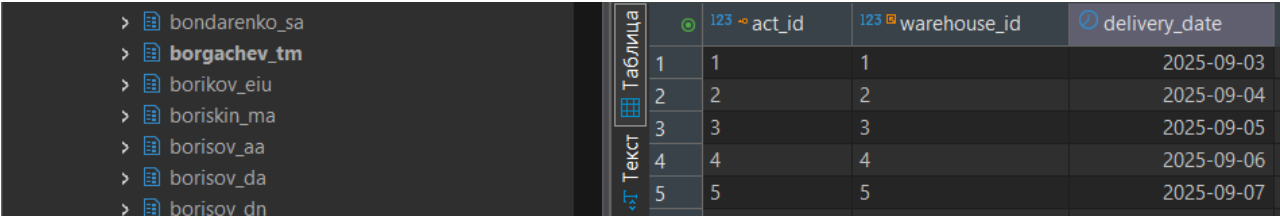
Таблица contract представлена на Рисунке 8.



	123 contract_id	123 supplier_id	contract_date	A-Z document_link
1	1	1	2024-09-08	https://contracts.example.com/1
2	2	2	2024-09-08	https://contracts.example.com/2
3	3	3	2024-09-08	https://contracts.example.com/3
4	4	4	2024-09-08	https://contracts.example.com/4
5	5	5	2024-09-08	https://contracts.example.com/5
6	6	6	2024-09-08	https://contracts.example.com/6

Рисунок 8 —Таблица contract

Таблица delivery_act представлена на Рисунке 9.



	123 act_id	123 warehouse_id	delivery_date
1	1	1	2025-09-03
2	2	2	2025-09-04
3	3	3	2025-09-05
4	4	4	2025-09-06
5	5	5	2025-09-07

Рисунок 9 —Таблица delivery_act

Таблица material_order представлена на Рисунке 10.

	order_id	raw_material_id	employee_id	supplier_id	contract_id	order_date	
1	1	1	3	1	1	2025-09-03	1
2	2	2	3	1	1	2025-09-03	1
3	3	1	3	1	1	2025-09-03	1
4	4	2	3	1	1	2025-09-03	1
5	5	1	3	1	1	2025-09-03	1
6	6	2	3	1	1	2025-09-03	1
7	7	1	3	1	1	2025-09-07	5
8	8	1	3	1	1	2025-09-07	5
9	9	1	3	1	1	2025-09-07	5
10	10	1	3	1	1	2025-09-07	5
11	11	1	3	1	1	2025-09-07	5
12	34	1	3	1	1	2024-01-15	1
13	35	2	3	2	2	2024-01-20	2
14	36	3	2	3	3	2024-02-05	3
15	37	4	3	4	4	2024-02-10	4
16	38	5	2	5	5	2024-02-15	5
17	39	6	3	1	1	2024-03-01	1
18	40	7	2	2	2	2024-03-05	2
19	41	8	3	3	3	2024-03-10	3
20	42	9	2	4	4	2024-03-15	4
21	43	10	3	5	5	2024-04-01	5
22	44	11	2	1	1	2024-04-05	1
23	45	12	3	2	2	2024-04-10	2

Рисунок 10 —Таблица material_order

Таблица detail_type представлена на Рисунке 11.

	detail_type_id	name
1	1	Металл
2	2	Пластик
3	3	Химикаты
4	4	Дерево
5	5	Текстиль
6	6	Электронные
7	7	Турбина
8	8	Сталь

Рисунок 11 —Таблица detail_type

Таблица detail представлена на Рисунке 12.

	detail_id	detail_type_id	name
1	1	1	Деталь 1
2	2	2	Деталь 2
3	3	7	Турбина ГТД
4	4	8	Стальная дета

Рисунок 12 —Таблица detail

Таблица detail_material представлена на Рисунке 13.

	detail_id	raw_material_id	amount
1	4	1	10,5
2	1	1	5,2
3	1	2	8,7
4	2	4	12,3

Рисунок 13 —Таблица detail_material

Таблица inventory_transaction представлена на Рисунке 14.

	transaction_id	inventory_id	employee_id	detail_id	amount_spent	transaction_date
1	1	1	3	4	50	2025-10-27 10:00:17.465
2	2	2	3	1	30	2025-10-27 10:00:17.465
3	3	5	6	2	25,5	2025-10-27 10:00:17.465
4	4	7	3	[NULL]	10	2025-10-27 10:00:17.465

Рисунок 14 — Таблица inventory_transaction

Запрос с созданием модифицируемого представления представлен в Листинге 1.

Листинг 1 — Создание модифицируемого представления

```
-- Задание №1: Модифицируемое представление для материалов типа "Металл"
CREATE OR replace VIEW borgachev_tm.metal_materials AS
SELECT raw_material_id, name, raw_material_type_id
FROM borgachev_tm.raw_material
WHERE raw_material_type_id = (
    SELECT raw_material_type_id
    FROM borgachev_tm.raw_material_type
    WHERE name = 'Металл'
);
```

Результат запроса представлен на Рисунке 15.

Name	Value
Updated Rows	0
Execute time	0.174s
Start time	Wed Oct 22 12:54:41 MSK 2025
Finish time	Wed Oct 22 12:54:42 MSK 2025
Query	-- Задание №1: Модифицируемое представление для материалов типа "Металл" CREATE OR replace VIEW borgachev_tm.metal_materials AS SELECT raw_material_id, name, raw_material_type_id FROM borgachev_tm.raw_material WHERE raw_material_type_id = (SELECT raw_material_type_id FROM borgachev_tm.raw_material_type WHERE name = 'Металл')

Рисунок 15 — Результат создания представления

Проверка создания представления представлена на Рисунке 16.

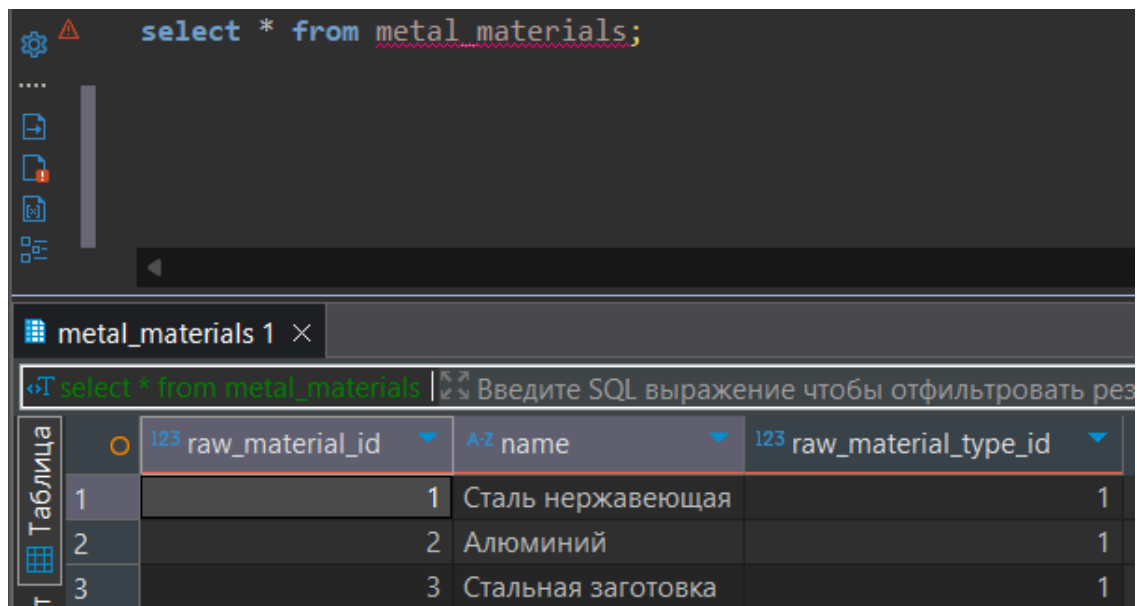


Рисунок 16 — Проверка содержимого представления

Задание №2: модификация данных через представление

Запрос с модификацией данных через представление представлен в Листинге 2.

Листинг 2 — Модификация данных через представление

```
-- Задание №2: Вставка новой записи через представление
-- Добавляем "Титан" как металл
INSERT INTO borgachev_tm.metal_materials (raw_material_type_id, name)
VALUES (1, 'Титан');
select * from metal_materials;
```

Результат модификации представлен на Рисунке 17.

metal_materials 1 ×			
select * from metal_materials			
Таблица	raw_material_id	name	raw_material_type_id
1	1	Сталь нержавеющая	1
2	2	Алюминий	1
3	3	Стальная заготовка	1
4	35	Титан	1

Рисунок 17 — Результат добавления материала в представление

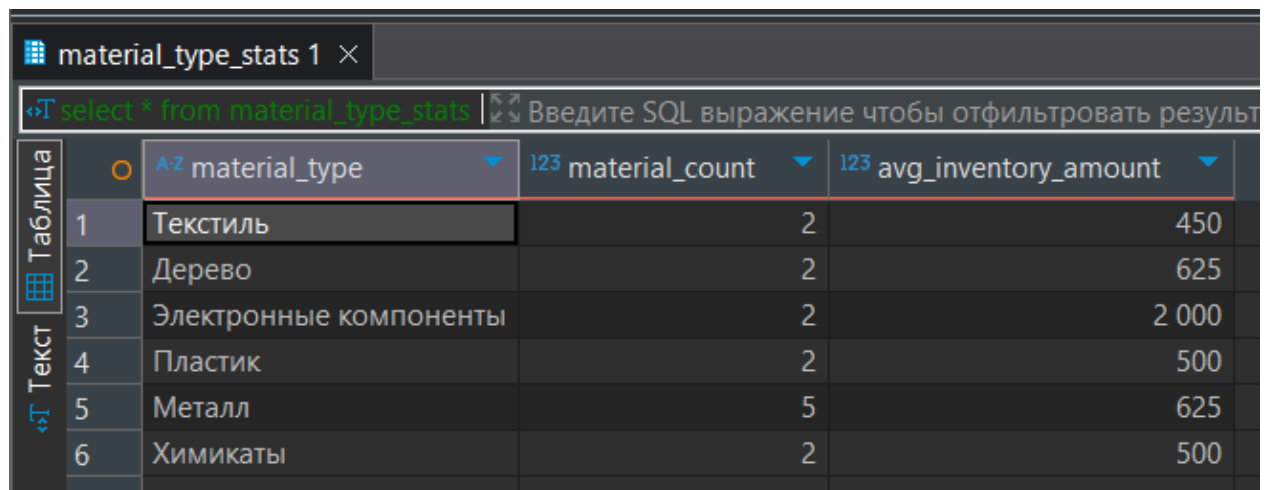
Задание №3: создание немодифицируемого аналитического представления

Запрос с созданием немодифицируемого представления представлен в Листинге 3.

Листинг 3 — Создание немодифицируемого представления

```
-- Задание №3: Аналитическое представление — статистика по типам материалов
CREATE VIEW borgachev_tm.material_type_stats AS
SELECT
    rmt.name AS material_type,
    COUNT(rm.raw_material_id) AS material_count,
    COALESCE(AVG(inv.amount), 0) AS avg_inventory_amount
FROM borgachev_tm.raw_material_type rmt
LEFT JOIN borgachev_tm.raw_material rm ON rmt.raw_material_type_id =
rm.raw_material_type_id
LEFT JOIN borgachev_tm.inventory inv ON rm.raw_material_id = inv.raw_material_id
GROUP BY rmt.raw_material_type_id, rmt.name;
select * from material_type_stats;
```

Результат создания отображения представлен на Рисунке 18.



	material_type	material_count	avg_inventory_amount
1	Текстиль	2	450
2	Дерево	2	625
3	Электронные компоненты	2	2 000
4	Пластик	2	500
5	Металл	5	625
6	Химикаты	2	500

Рисунок 18 — Немодифицируемое представление

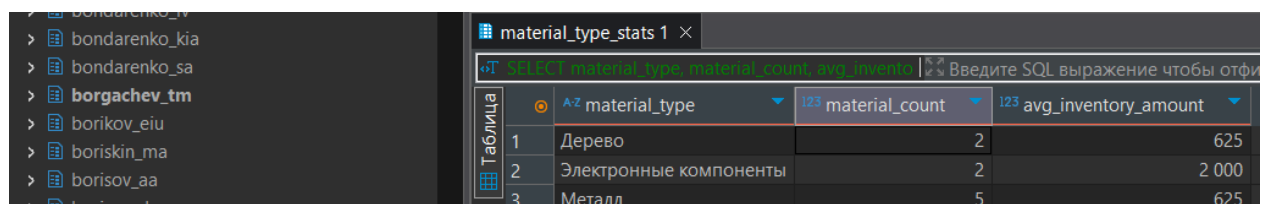
Задание №4: использование аналитического представления в запросах

Запрос с использованием аналитического представления представлен в Листинге 4.

Листинг 4 — Фильтрация по аналитическому представлению

```
-- Задание №4: Фильтрация по аналитическому представлению
SELECT material_type, material_count, avg_inventory_amount
FROM borgachev_tm.material_type_stats
WHERE avg_inventory_amount > 500;
```

Результат запроса представлен на Рисунке 19.



	material_type	material_count	avg_inventory_amount
1	Дерево	2	625
2	Электронные компоненты	2	2 000
3	Металл	5	625

Рисунок 19 — Результат запроса с аналитическим представлением

Задание №5: Создание и обновление материализованного представления

Запрос с созданием материализованного представления представлен в Листинге 5.

Листинг 5 — Материализованное представление

```
-- Задание №5.1: Материализованное представление
-- Создаём материализованное представление: суммарное количество заказанного сырья по
каждому поставщику
CREATE MATERIALIZED VIEW borgachev_tm.supplier_material_totals AS
SELECT
    s.supplier_id,
    s.name AS supplier_name,
    rm.name AS raw_material_name,
    SUM(mo.raw_material_id) FILTER (WHERE mo.raw_material_id IS NOT NULL) AS
order_count,
    COUNT(mo.raw_material_id) AS total_orders,
    COUNT(*) AS total_order_lines
FROM borgachev_tm.supplier s
JOIN borgachev_tm.material_order mo ON s.supplier_id = mo.supplier_id
JOIN borgachev_tm.raw_material rm ON mo.raw_material_id = rm.raw_material_id
GROUP BY s.supplier_id, s.name, rm.name;
select * from supplier material totals;
```

Результат запроса представлен на Рисунке 20.

supplier_id	supplier_name	raw_material_name	order_count	total_orders	total_order_lines
1	МеталлПром	Кислота серная	6	1	1
2	ТекстильГрупп	Хлопок натуральный	10	1	1
3	ЛесРесурс	Фанера березовая	9	1	1
4	МеталлПром	Полиэстер	11	1	1
5	МеталлПром	Алюминий	6	3	3
6	ЛесРесурс	Полиэтилен	4	1	1
7	МеталлПром	Сталь нержавеющая	9	9	9
8	ХимТорг	Доска дубовая	8	1	1
9	ТекстильГрупп	Полипропилен	5	1	1
10	ПластикИндустрия	Микросхемы	12	1	1
11	ПластикИндустрия	Алюминий	2	1	1
12	ХимТорг	Стальная заготовка	3	1	1
13	ПластикИндустрия	Едкий натр	7	1	1

Рисунок 20 — Результат создания материализованного представления

Запрос с обновлением материализованного представления представлен в Листинге 6.

Листинг 6 — Обновление представления

```
-- Задание 5.2: обновление представления
INSERT INTO borgachev_tm.material_order
    (raw_material_id, employee_id, supplier_id, contract_id, order_date, act_id)
VALUES ((SELECT raw_material_id FROM borgachev_tm.raw_material WHERE name = 'Сталь
нержавеющая'), 3, 1, 1, CURRENT_DATE, 1);
REFRESH MATERIALIZED VIEW borgachev_tm.supplier_material_totals;
select * from supplier material totals;
```

Результат обновления представления отображен на Рисунке 21.

supplier_id	supplier_name	raw_material_name	order_count	total_orders	total_order_lines
1	МеталлПром	Кислота серная	6	1	1
2	ТекстильГрупп	Хлопок натуральный	10	1	1
3	ЛесРесурс	Фанера березовая	9	1	1
4	МеталлПром	Полиэстер	11	1	1
5	МеталлПром	Алюминий	6	3	3
6	ЛесРесурс	Полиэтилен	4	1	1
7	МеталлПром	Сталь нержавеющая	10	10	10
8	ХимТорг	Доска дубовая	8	1	1
9	ТекстильГрупп	Полипропилен	5	1	1
10	ПластикИндустрия	Микросхемы	12	1	1
11	ПластикИндустрия	Алюминий	2	1	1
12	ХимТорг	Стальная заготовка	3	1	1
13	ПластикИндустрия	Едкий натр	7	1	1

Рисунок 21 — Обновленное представление

Задание №6: разработка пользовательской функции для аналитических вычислений

Запрос с разработкой пользовательской функции для аналитических вычислений представлен в Листинге 7.

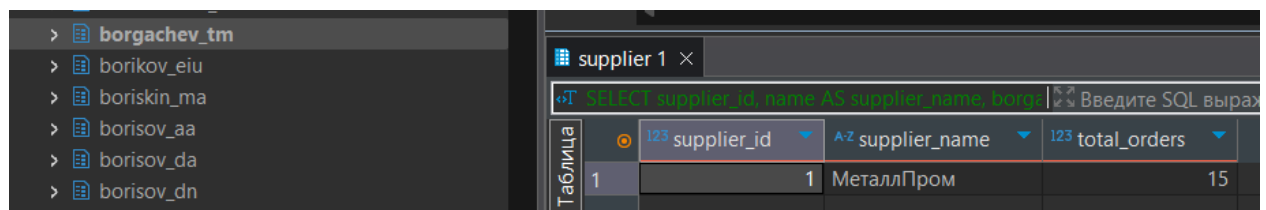
Листинг 7 — Создание функции

```
-- Задание №6: Функция: возвращает общее число заказов материалов от указанного поставщика
CREATE OR REPLACE FUNCTION borgachev_tm.get_supplier_order_count(target_supplier_id
BIGINT)
RETURNS BIGINT AS $$
DECLARE
    total_orders BIGINT;
BEGIN
    SELECT COUNT(*)
    INTO total_orders
    FROM borgachev_tm.material_order
    WHERE supplier_id = target_supplier_id;

    RETURN COALESCE(total_orders, 0);
END;
$$ LANGUAGE plpgsql;

-- Пример вызова: получить количество заказов для поставщика с ID = 1 (МеталлПром)
SELECT
    supplier_id,
    name AS supplier_name,
    borgachev_tm.get_supplier_order_count(supplier_id) AS total_orders
FROM borgachev_tm.supplier
WHERE supplier_id = 1;
```

Результат запроса с вызовом функции представлен на Рисунке 22.



supplier 1 ×			
SQL SELECT supplier_id, name AS supplier_name, borgachev_tm.get_supplier_order_count(supplier_id) AS total_orders FROM borgachev_tm.supplier WHERE supplier_id = 1			
Таблица	supplier_id	supplier_name	total_orders
1	1	МеталлПром	15

Рисунок 22 — Результат работы функции

Задание №7: разработка хранимой процедуры для выполнения сложной операции

Запрос с разработкой хранимой процедуры для выполнения сложной операции представлен в Листинге 8.

Листинг 8 — Создание процедуры

```
-- Задание №7: Процедура: добавление/обновление материала на складе
CREATE OR REPLACE PROCEDURE borgachev_tm.add_inventory(
    p_raw_material_name TEXT,
    p_raw_material_type_name TEXT,
    p_warehouse_id BIGINT,
    p_amount NUMERIC(10,2),
    OUT result_status TEXT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_raw_material_id BIGINT;
    v_raw_material_type_id BIGINT;
    v_existing_type_id BIGINT;
    v_existing_type_name TEXT;
    v_warehouse_exists BOOLEAN;
    v_current_amount NUMERIC(10,2);
BEGIN
    -- Проверяем, существует ли склад
    SELECT EXISTS(SELECT 1 FROM borgachev_tm.warehouse WHERE warehouse_id = p_warehouse_id) INTO v_warehouse_exists;
    IF NOT v_warehouse_exists THEN
        result_status := 'ERROR: Warehouse does not exist';
        RETURN;
    END IF;

    -- Проверяем, существует ли материал и получаем его тип
    SELECT rm.raw_material_id, rm.raw_material_type_id
    INTO v_raw_material_id, v_existing_type_id
    FROM borgachev_tm.raw_material rm
    JOIN borgachev_tm.raw_material_type rmt ON rm.raw_material_type_id = rmt.raw_material_type_id
    WHERE rm.name = p_raw_material_name;

    -- Если материал существует, проверяем соответствие типа
    IF v_raw_material_id IS NOT NULL THEN
        SELECT rmt.name INTO v_existing_type_name
        FROM borgachev_tm.raw_material_type rmt
        WHERE raw_material_type_id = v_existing_type_id;

        -- Сравниваем переданный тип с существующим
        IF v_existing_type_name != p_raw_material_type_name THEN
            result_status := 'ERROR: Material type mismatch. New type is not the same as old type';
            RETURN;
        END IF;

        -- Типы совпадают, используем существующие ID
        v_raw_material_type_id := v_existing_type_id;
    ELSE
        -- Материал не существует, создаем новый тип если нужно и сам материал
        SELECT raw_material_type_id INTO v_raw_material_type_id
        FROM borgachev_tm.raw_material_type
        WHERE name = p_raw_material_type_name;

        IF v_raw_material_type_id IS NULL THEN
            INSERT INTO borgachev_tm.raw_material_type (name) VALUES (p_raw_material_type_name)
            RETURNING raw_material_type_id INTO v_raw_material_type_id;
        END IF;

        INSERT INTO borgachev_tm.raw_material (raw_material_type_id, name)
        VALUES (v_raw_material_type_id, p_raw_material_name);
    END IF;
END;
```

```

RETURNING raw_material_id INTO v_raw_material_id;

-- Создаем новую запись в inventory
INSERT INTO borgachev_tm.inventory (raw_material_id, warehouse_id, amount)
VALUES (v_raw_material_id, p_warehouse_id, p_amount);

result_status := 'SUCCESS: New material added and inventory created';
RETURN;
END IF;

-- Материал уже существует с правильным типом, обновляем количество на складе
-- Блокируем строку для предотвращения гонок
SELECT amount INTO v_current_amount
FROM borgachev_tm.inventory
WHERE raw_material_id = v_raw_material_id AND warehouse_id = p_warehouse_id
FOR UPDATE;

IF v_current_amount IS NULL THEN
    -- Запись в inventory не существует, создаем новую
    INSERT INTO borgachev_tm.inventory (raw_material_id, warehouse_id, amount)
    VALUES (v_raw_material_id, p_warehouse_id, p_amount);
ELSE
    -- Обновляем существующую запись
    UPDATE borgachev_tm.inventory
    SET amount = amount + p_amount
    WHERE raw_material_id = v_raw_material_id AND warehouse_id = p_warehouse_id;
END IF;

result_status := 'SUCCESS: Inventory updated';
END;
$$;

```

Результат создания представлен на Рисунке 23.

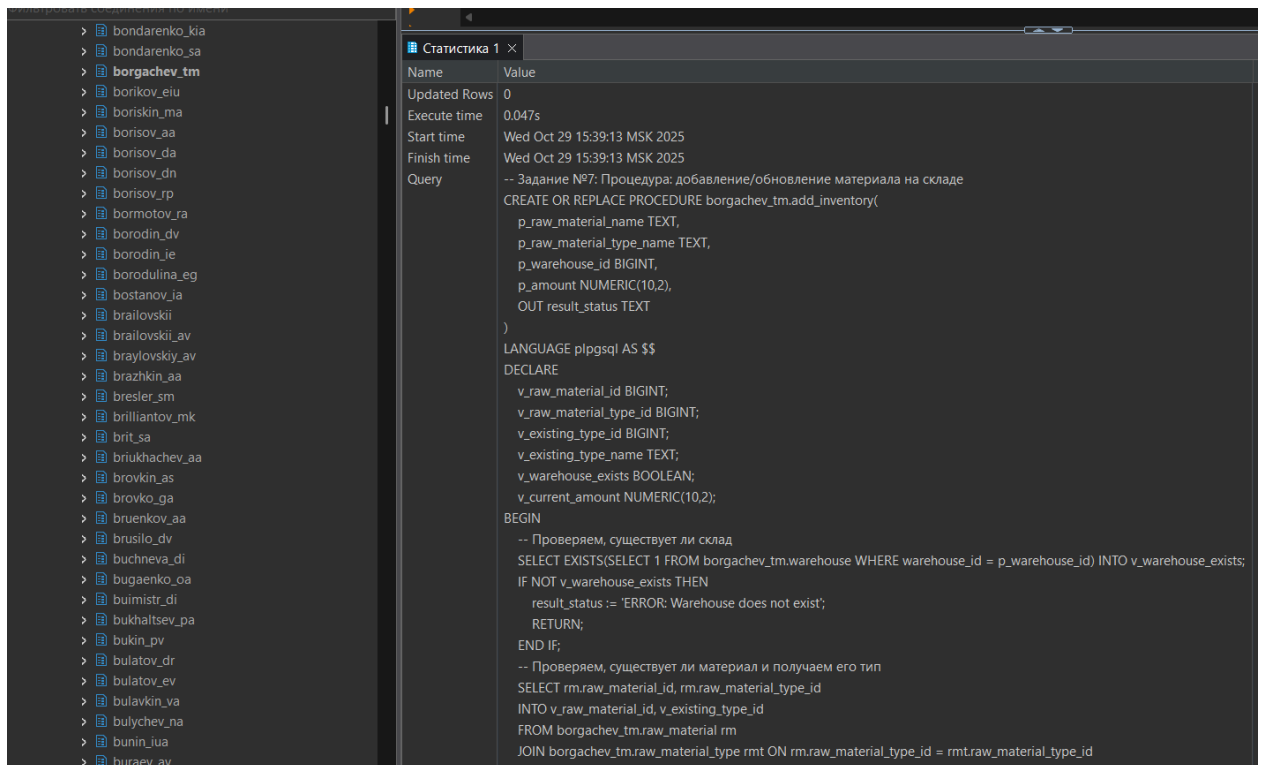


Рисунок 23 — Результат создания процедуры

Задание №8: демонстрация вызова хранимой процедуры

Запрос с успешной демонстрацией вызова хранимой процедуры представлен в

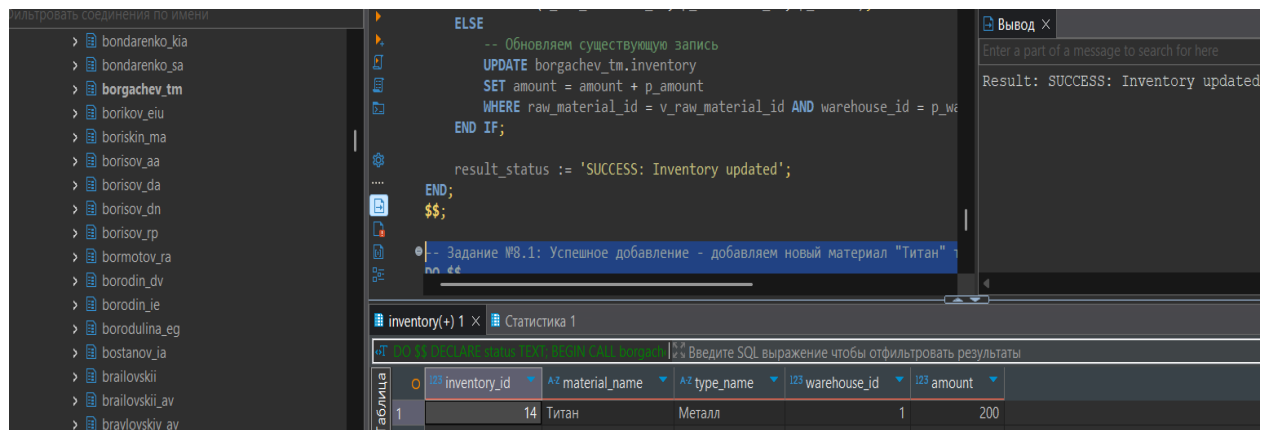
Листинге 9.

Листинг 9 — Корректный вызов процедуры

```
-- Задание №8.1: Успешное добавление - добавляем новый материал "Титан" типа "Металл"
на склад 1
DO $$
DECLARE
    status TEXT;
BEGIN
    CALL borgachev_tm.add_inventory('Титан', 'Металл', 1, 200, status);
    RAISE NOTICE 'Result: %', status;
END $$;

-- Проверяем результат
SELECT i.inventory_id, rm.name as material_name, rmt.name as type_name, w.warehouse_id, i.amount
FROM borgachev_tm.inventory i
JOIN borgachev_tm.raw_material rm ON i.raw_material_id = rm.raw_material_id
JOIN borgachev_tm.raw_material_type rmt ON rm.raw_material_type_id = rmt.raw_material_type_id
JOIN borgachev_tm.warehouse w ON i.warehouse_id = w.warehouse_id
WHERE rm.name = 'Титан';
```

Результат запроса представлен на Рисунке 24.



The screenshot shows a database IDE with a project tree on the left containing various schema objects. The main editor displays a PL/SQL block with an UPDATE statement and a RAISE NOTICE. The output window on the right shows the message 'Result: SUCCESS: Inventory updated'. Below the editor, a table titled 'inventory(+)' shows the result of a query, with one row containing the values 14, Титан, Металл, 1, and 200.

inventory_id	material_name	type_name	warehouse_id	amount
14	Титан	Металл	1	200

Рисунок 24 — Успешный вызов процедуры

Запрос с неудачной демонстрацией вызова хранимой процедуры представлен в Листинге 10.

Листинг 10 — Попытка списать больше, чем есть

```
-- Задание №8.2: Неудачная попытка - склад не существует (warehouse_id = 999)
DO $$
DECLARE
    status TEXT;
BEGIN
    CALL borgachev_tm.add_inventory('Золото', 'Металл', 999, 50, status);
    RAISE NOTICE 'Result: %', status;
END $$;
```

Результат запроса представлен на Рисунке 25.

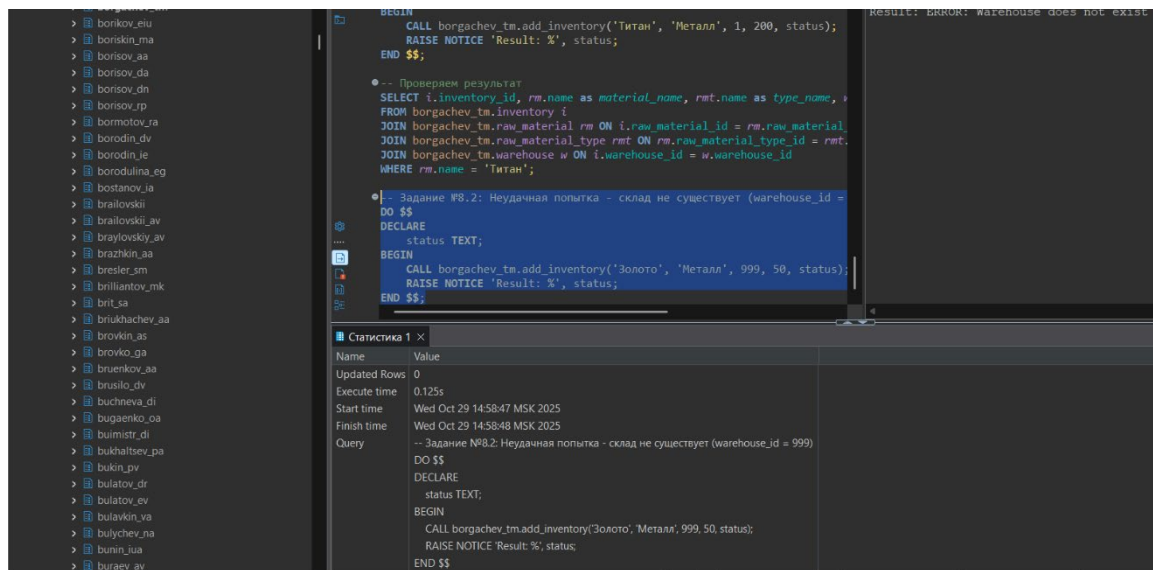


Рисунок 25 — Неудачный результат вызова процедуры